WHITE PAPER

The Connector as Service

A reasoned, honest case for F1R3FLY's strategy — and the pattern it hands its clients

F1R3FLY.io

July 4, 2026 · Version 1.0

Abstract. F1R3FLY is a distributed-computing platform that resolves a long-standing trade-off between security and scale. It delivers the Byzantine-grade security of decentralized networks — on which the authority-escalation attacks that recently crippled Marks & Spencer and Jaguar Land Rover have no path to run — while scaling with added hardware in the manner of the hyperscale clouds. This paper makes the business case that follows from that substrate. We argue that value in networked systems increasingly accrues not at the endpoints but at the *connectors* that route activity across community boundaries; that community fragmentation makes such cross-boundary traffic a growth industry rather than a thin slice; and that a connector aimed at *service* rather than *capture* defends itself by staying good. We show that the resulting pattern — assemble a collection of communities, then build the means to connect them — is scale-invariant, recurring from the protocol layer through F1R3FLY's substrate to every application built on it, and already being executed by clients across unrelated domains. We are explicit throughout about what is established and what remains to be measured.

Executive Summary

- **Come for security, stay for programmability.** Chief information-security officers at Fortune 100 companies are alarmed — and rightly. In 2025 a single social-engineered credential brought Marks & Spencer to critical operational failure, and a comparable intrusion halted Jaguar Land Rover for weeks and drew an unprecedented UK government backstop. Neither attack broke any cryptography. Both were pure authority escalation. That is the failure mode F1R3FLY is built to remove.
- **The security gap F1R3FLY closes.** Crash fault-tolerant systems (AWS, Google Cloud, Azure, OCI) *scale* with hardware but are not secure against an adversary operating from inside. Byzantine fault-tolerant systems (Bitcoin, Ethereum, Solana) *are* secure — chiefly because there is no authority-escalation path to seize — but they get *slower* as hardware is added. F1R3FLY delivers Byzantine-grade security *with* crash-tolerant scaling: it gets faster as hardware is added. (Earlier benchmarks showed near-linear scaling into the 100,000-tps range; re-validation with independent third parties is underway.)
- **Three layers, not one.** Security here is not tokens on top of cryptography. (1) The rho calculus is at its core an *object-capability* model: no ambient authority to escalate into. (2) *Tokens amplify and attenuate* capabilities, giving least-privilege authority enforced by the runtime. (3) *Spatial-behavioral types* give up-front static guarantees — and, the same mechanism, a new kind of search.
- **Rholang = storage + query.** Rholang derives its semantics directly from the rho calculus. Its channels abstract uniformly from a memory location up to an entire web

service, so `channel!(data)` stores anywhere and the for-comprehension queries by *behavior*: `SELECT` pattern `FROM` channel `WHERE` spatial-behavioral-condition `DO` program — the relational and no-SQL worlds in one primitive, over a strictly larger query space than SQL.

- **The model retraces the Internet — and repeats at every layer.** The Internet lowered the cost of networked communities but omitted programmability, storage, search, and security; F1R3FLY supplies exactly those four. Shards are the intranet phase (open-source, one checkbox to stand up); connector shards are the connectivity phase, where value accrues on transactions crossing between communities. The pattern is *scale-invariant*: a client first assembles a collection of communities, then builds its own means of connecting them — the same two phases, one level up, in its own domain. We pitched this to Weve (rolling out David Spivak's Plausible Fiction); they immediately saw the sense in it. It generalizes to job markets, content-creation markets, logistics, and more.
- **Why the demand is real.** A bet on *fragmentation, not consolidation*. Communities form by forking, and forks stay tied to their parents (citation, recruitment, differentiation, trade). Fragmentation manufactures cross-boundary edges at least as fast as nodes, so cross-network communication is a growth industry, not a thin slice.
- **Service, not capture.** Capture and enshittification are one mechanism seen at two times. We keep the exit open (open substrate) so the connector must *stay good* to hold its position. Two invariants keep it honest: **payer = served** (monetize realized match value, never surfacing priority) and **prediction suggests, never authors** (a model may propose a sentiment; only a human's mark is data).
- **Where the moat sits.** Not in the open-source engine. In *reach*, in *compounding interaction signal* (open sentiment), and in *application-layer matching* that aggregates two-sided liquidity. We expect connector shards to become a competitive market; our lead is that no one is yet looking here, plus our ability to ship new infrastructure such as automatic shard splitting.
- **This is already happening.** The strategy is being executed by real clients across unrelated domains: the Artificial Superintelligence Alliance (ASIChain, with SingularityNET), Boring Financial (gold-backed digital assets), and Weve (Plausible Fiction). Distribution runs through systems integrators — we have landed Tata, the integrator whose outsourced IT sat at the center of both the M&S and Jaguar Land Rover breaches, as a channel to roll out shards at enterprise scale.
- **What we still owe ourselves.** The realized cross-boundary fraction, take-rate stacking, local-vs-global network effects, and governance of a fee-bearing chokepoint are stated plainly in the final section. The case is strong; it is not yet a spreadsheet.

1 What F1R3FLY is, and why CISOs are listening

The fastest way into this business is through fear, because the fear is justified and current. In April 2025 Marks & Spencer — a company with over a thousand stores and sixty-odd thousand staff — was brought to critical operational failure by a ransomware attack. The attackers did not defeat encryption. They telephoned a third-party IT helpdesk, impersonated an employee, and had a password reset performed for them; from that toehold they extracted the corporate Active Directory credential database, cracked the hashes offline, moved laterally until they held domain-wide authority, and then encrypted the estate. Online orders were suspended for roughly six weeks; stores reverted to pen and paper; the profit impact ran to several hundred

million pounds [14, 15]. A few months later a comparable intrusion halted Jaguar Land Rover's production across its plants for five to six weeks, propagated to thousands of downstream suppliers, and inflicted an estimated £1.9 billion in economic damage — the most costly cyber event in UK history [17]. The government stepped in with an unprecedented loan guarantee of up to £1.5 billion, the first time it had financially backstopped a company after a cyberattack [16]. Ministers framed it as protecting jobs; observers called it a bailout and warned it signals that some firms are now too big to fail. Honestly, both readings are true, and both make the same point: a single escalated credential can now threaten a national-scale enterprise.

The structural lesson is the one that matters for F1R3FLY. *No cryptography was broken in either case.* The breaches worked by *authority escalation*: turn a small foothold into ambient, domain-wide power. That is not an accident of these two incidents; it is the native weakness of the architecture almost every large enterprise runs on.

The scaling–security dilemma

Distributed systems have, until now, forced a choice.

Crash fault-tolerant systems — the hyperscale clouds, AWS, Google Cloud, Azure, OCI, and the coordination protocols beneath them — assume components fail by *stopping*, not by turning against you. They scale beautifully: add hardware, get throughput. But they have no defense against a participant that has been taken over and acts maliciously from the inside. Once an attacker escalates to an authoritative account or node, the system trusts them completely. This is exactly the path M&S and JLR were walked down.

Byzantine fault-tolerant systems — Bitcoin, Ethereum, Solana — assume some participants may be arbitrarily malicious and reach agreement anyway [5]. They are secure, predominantly because there is *no authority-escalation path for a hacker to seize*: there is no admin account that owns the network, and compromising one node yields one node, not the whole. But classic Byzantine agreement pays for this with all-to-all communication [6], so throughput *degrades* as nodes are added. They get slower with more hardware — the opposite of what an enterprise needs.

> F1R3FLY's proposition is to end the trade-off: **Byzantine-grade security with crash-tolerant scaling.** It retains the property that there is no ambient authority to escalate into, and it *scales as hardware is added* — getting faster, like the clouds, rather than slower, like the chains. Fortune 100 companies come for that security. They stay for the programmability and for the distributed, decentralized features that the same substrate makes available.

What makes this possible is a best-of-breed model of concurrent computation: the *rho calculus* (Reflective Higher-Order calculus) [1]. It is not a bolt-on. Rholang, F1R3FLY's smart-contract language, and RSpace, its storage-and-execution mechanism, both derive their semantics *directly* from the rho calculus. Security, storage, execution, and query are therefore not four subsystems bolted together — they are four faces of one mathematical object.

Where the numbers stand

Earlier benchmarks bear the scaling claim out and show its shape. A single-processor node sustained on the order of 1,000 transactions per second. Throughput then scaled close to linearly with the network: reaching roughly 10,000 tps took a network of about ten nodes, provisioned at around ten cores each; reaching roughly 100,000 tps took about one hundred nodes at around one hundred cores each. The pattern is the one that matters — you buy throughput by adding hardware, nodes to the network and cores to the node — which is precisely the crash-tolerant

scaling behavior, achieved here *without* surrendering Byzantine security. It is worth restating the contrast: a classical Byzantine chain would have gone the other way, slowing as those nodes were added.

The mechanism underneath is the rho calculus's native concurrency, and a more recent result makes it concrete: parallel compositions occupy cores evenly. A par of sixteen reader/writer pairs spreads across sixteen cores, each core carrying, on average, one of the transactions. Parallelism written in the program becomes parallelism on the hardware without hand-tuning — which is exactly why adding cores adds throughput rather than contention.

Two honest caveats belong with these figures. First, they are *earlier* numbers, and the system has changed underneath them: recent work — cost accounting among it — has removed much of the block-merge machinery, which we expect to move the results (we believe favorably, but that is to be measured, not asserted). Second, and for that reason, we are redoing the benchmarks and working with third parties for *independent* confirmation rather than resting on our own measurements. The claim we stand behind today is therefore the honest one: earlier benchmarks showed close-to-linear scaling into the hundred-thousand-tps range, and re-validation is in progress. A skeptical CISO should be handed the independent numbers when they land.

2 Three layers of security

The security story is often mis-told as "cryptography, plus tokens on top." That undersells it by two full layers. F1R3FLY stacks three distinct guarantees, and the deepest one is precisely the one the 2025 breaches would have had to defeat and could not.

1. **The rho calculus is an object-capability model [3, 4].** There is no ambient authority. To act on something you must hold an *unforgeable reference* to it — a name, a channel — and you cannot name what you were never given. There is no global "domain," no super-user, no ambient scope to escalate into. Compromising a principal yields exactly the capabilities that principal held and *nothing more*; authority does not amplify by default. Re-read the M&S kill chain against this: impersonate the helpdesk, reset a credential, and in a conventional system you are on a path toward domain admin — ambient authority over everything. In an object-capability system that path does not exist, because there is no "everything" to become. The stolen foothold is worth only what that one holder could already do. This is the structural elimination of the attack, not a patch against it.

2. **Tokens amplify and attenuate capabilities.** On top of the capability core, tokens act as capability *modulators*. They *attenuate* — handing a strictly weaker capability: read-only, rate-limited, time-boxed, scoped to a single channel — and they *amplify* — unlocking a stronger action only when the right tokens are jointly presented. The result is least-privilege authority that is *composable* and enforced by the runtime rather than promised in a policy document. (Located token stacks pin authority to specific interaction channels, closing the ambient-authority gaps that would otherwise creep back in.) Where conventional security asks operators to configure least privilege correctly and punishes every lapse, here least privilege is the default shape of the system.

3. **Spatial-behavioral types give up-front static guarantees.** Before a line of code runs, its type can constrain not just the shape of its data but its *spatial structure* (what is connected to what) and its *behavior* (how it will interact). A CISO can be given a static guarantee that a contract cannot reach a channel it was not granted, cannot exfiltrate to an unlisted destination, cannot exhibit a forbidden interaction — checked at compile time, not hoped for at runtime.

The hinge. The third security layer is also the discovery engine. The very same spatial-behavioral types that let a security officer *prove* how code will behave are what let a community *query* for how code (or content, or a contract) behaves. Up-front safety and behavioral search are one mechanism viewed from two directions. This is why "come for security, stay for programmability" is literally true at the type level: the feature that sells to the CISO is the feature that powers everything in the rest of this note.

3 Rholang, through its rho calculus core

Because Rholang inherits its semantics from the rho calculus, a very small vocabulary does a great deal of work. The central abstraction is the *channel*. A channel is a name you can send to and receive on — and, crucially, it abstracts *uniformly* across scales: the same construct denotes a memory location, a database row, a queue, a service endpoint, or an entire web service. You do not change primitives as you move from in-memory state to a remote service; you change what a channel points at.

That single abstraction gives Rholang both halves of the modern data stack. Storage is trivial: `channel!(data)` puts anything anywhere — the no-SQL "put it wherever, shape it however" ergonomics. Query is the same primitive read backwards. The for-comprehension is, in a different syntax, a database query:

`SELECT` *pattern* `FROM` *channel* `WHERE` *spatial-behavioral-condition* `DO` *program*.

And here Rholang exceeds SQL rather than merely matching it. SQL's `WHERE` ranges over data *values*. Rholang's ranges over spatial-behavioral conditions — so you can ask for processes that will *behave* a certain way, not merely data shaped a certain way. That is a strictly larger query space, and it is the query space no relational engine can express. Rholang thus offers the query discipline of the relational world and the placement freedom of the no-SQL world at once, because both are just directions of use of one channel primitive whose meaning comes from the rho calculus.

One multiplier turns this from a language feature into a platform strategy. OSLF (Operational Semantics in Logical Form) [2] is a functor over every language definable in the system: for each such language it *generates* the query logic — at the type level, pricing included — rather than requiring it to be hand-built. Every service built on F1R3FLY therefore *inherits* behavioral query and pricing instead of engineering them. We return to what that makes possible in §7.

4 The business model, and the pattern it repeats

The Internet did not connect billions of devices in a single stroke, and it did not begin by asking everyone to trust a global network. It began *locally*. University departments, small businesses, and government offices built *intranets* — self-contained networks that delivered value inside one organization, where the trust boundary was small and comprehensible. Only once intranets were everywhere, and their internal value proven, did the value of connecting them *between* organizations grow large enough to overwhelm the entirely reasonable fears about security that had kept them separate. Connectivity won because there was, by then, an enormous installed base with an obvious reason to reach each other.

The Internet's protocols were the enabling move: they lowered the cost of creating networked communities, and that was their triumph. But they left a great deal out. Programmability, storage, search, and security were never first-class, uniform properties of the fabric — you got packets and pages, addressing and transport, not a computational, queryable, secure substrate. Every application has had to rebuild those four things, badly and separately, ever since; the

2025 breaches are what "security bolted on afterward" looks like at scale. F1R3FLY supplies exactly the missing layer. A shard *is* a networked community with programmability (Rholang), storage (RSpace), behavioral search, and Byzantine-grade security built in rather than bolted on.

F1R3FLY retraces exactly this path.

- **The intranet phase** is the *shard*. Open-source, and so easy to stand up that friction is no longer a reason to hesitate — a single checkbox at setup registers a new shard with the connector. A community gets immediate, self-contained value: secure computation, storage, and behavioral search over its own assets, inside its own trust boundary.
- **The connectivity phase** is the *connector shard*. When one shard holds something another needs, they connect — transport through the F1R3FLY connector, and, one layer up, application-level brokers (PF brokers, and their analogues for other services) that let counterparts on different shards actually find each other. Value accrues on the small transactions that cross between communities.

We should be honest about the competitive shape of this. We do *not* expect to own the connector layer by fiat. As the pattern proves out, we expect connector shards to become a *market*, with competition — much as transit and interconnect became markets once the value of connecting was established. That expectation is not a weakness in the plan; it is the plan working. Our advantages in that market are three, and they are stated plainly so they can be tested:

- **Lead.** No one is currently looking in this direction. The connector-as-service thesis, and the behavioral-search and open-sentiment capabilities that differentiate a good connector from a bare pipe, are not yet on anyone else's roadmap.
- **Differentiation that does not commoditize.** The elements developed in the rest of this note — behavioral discovery, open sentiment, the storage flywheel — make a F1R3FLY connector qualitatively better at helping communities find each other, not merely cheaper at moving packets.
- **Infrastructure velocity.** We can roll out new substrate capabilities others cannot, such as *automatic shard splitting* — a shard that grows too large or too hot for one operator dividing itself, with connectivity preserved. Capabilities like this keep the lead compounding rather than eroding.

The pattern iterates

The pattern does not stop at F1R3FLY's own layer. It is the same pattern at every layer, and that is the heart of what F1R3FLY offers a client. Supplying the substrate leaves a great deal to be built above it — and what gets built above it has exactly the shape of what sits below. What a client builds, first, is a *collection of communities*. Then it builds its own *special means of bringing those communities together*. That is the intranet phase and the connector phase again, one level up, in the client's own domain.

This is not theoretical, and Weve is not the only client — nor the first — to have seen it. The Artificial Superintelligence Alliance (ASI) is building ASIChain on F1R3FLY and adopting this very strategy, with SingularityNET, one of the Alliance's members, deeply involved in the work. Boring Financial is building a gold-backed digital-asset solution on F1R3FLY. And Weve is rolling out David Spivak's Plausible Fiction system. Three clients in three domains that could hardly be less alike — decentralized superintelligence infrastructure, hard-asset finance, and two-sided fiction markets — executing the same pattern on the same substrate. That it reads

across domains this different is the strongest evidence we have that the pattern is real rather than a story we enjoy telling.

Weve is the cleanest worked example. The company was founded to roll out David Spivak's Plausible Fiction idea [10, 11], and when we described this approach they immediately saw the sense in it. What Weve builds first is a collection of PF communities, each on its own shard, each delivering value locally. Then Weve builds the thing only Weve can build: the means by which Plausible Fictions on different shards find each other, the PF brokers that interconnect, with Weve earning on cross-broker transactions. F1R3FLY gives Weve the substrate — a PF shard is trivial to stand up because programmability, storage, search, and security are already there — so Weve builds only the domain-specific connector that is its actual business.

Weve is one instance of a general form. Steve Knight's Our Thing does it for creators and their audiences: gather creator communities, then build the discovery that connects them. A job market does it: gather worker and employer communities, then build the matching that connects them. A content-creation market, a logistics network — each first assembles communities, then builds the connective tissue that is its differentiated value. In every case the substrate work, the part that is hard, generic, and unrewarding to rebuild, is what F1R3FLY provides; the connector, the part that is domain-specific and defensible, is what the client builds and owns.

> The pattern is *scale-invariant*. At the protocol layer, the Internet lowered the cost of networked communities but omitted programmability, storage, search, and security. At the substrate layer, F1R3FLY supplies those four and commoditizes standing a community up. At the application layer, each client assembles communities and then connects them. And inside each application, communities fork and the pattern begins again (§5). At every layer the shape is identical: build local value first, then earn at the connector that routes across boundaries. F1R3FLY's business is to provide the substrate that makes the pattern cheap to run at the layers above — and to hold the connector position at its own.

Distribution: the systems-integrator channel

Making a shard trivial to stand up removes the friction; it does not, by itself, get shards stood up. Distribution is its own problem, and our answer is deliberate: work through *systems integrators*, whose existing enterprise relationships and delivery capacity roll out new shards at a scale direct sales never could. The integrator already owns the client relationship, the deployment teams, and the trust; we supply the substrate they deploy. This is also the honest mitigation for the cold-start problem — shards are worth more connected, and an integrator's client base arrives pre-clustered into communities that already have reason to reach each other.

We have landed Tata as exactly this channel, and the fit is pointed. Tata's outsourced IT operations sat at the center of both 2025 incidents that open this note: the social-engineered helpdesk at Marks & Spencer, and Jaguar Land Rover's outsourced security function. We say this without finger-pointing — those were social-engineering attacks against processes, not a flaw peculiar to one integrator, and every large integrator runs the same exposed surface. The point is that Tata has learned the cost of the authority-escalation failure mode first-hand and at scale. That makes it both uniquely motivated to carry a substrate on which that failure mode has no path to run, and uniquely credible when it tells an enterprise buyer as much. The integrator that lived through the breach becomes the channel for the cure. That is the loop from §1 closing.

The rest of this note argues that the connectivity phase is a real and growing business (§5), that aiming it at service rather than capture is both right and defensible (§6), that its differentiator is

genuinely hard to copy (§7–§8), and — honestly — what remains to be quantified (§12).

5 Why the demand is real: fragmentation, not consolidation

Nearly every platform business bets on consolidation: aggregate everyone onto one substrate and win by becoming the terminal network. We bet the other way. The count of networks grows super-linearly, and the durable value is in the seams between them rather than inside any one.

The evidence is that communities form by *forking*, and forks do not sever the tie to what they grew from. US television went from three networks to thousands; networks now nucleate *inside* networks — an artist network growing inside YouTube, a science-content network growing inside the same host. Generations want their own voice; disciplines split faster than anyone can track. Set theory alone is a full-time job to follow, before one even reaches the condensed mathematics of Scholze and Clausen [13]. And these forks are never truly independent of the culture and networks they came from, however much they may wish to be.

That non-independence is the whole point, and it makes the demand a *flow* rather than a spike. When Bitcoin [7] forked, holders suddenly had a position on both chains, and exchanges earned fat fees as traders played both sides — but that clears once holders rebalance. A sub-discipline that forks from its parent does not settle once and stop. It keeps citing the parent, recruiting from the parent's audience, differentiating against it, and trading back into it, permanently. The umbilical is not cut; it is transacted across indefinitely.

> So: as long as one community holds something another needs, there is a source of cross-community transactions — and forking, the dominant mode of network formation, continuously manufactures exactly that condition. Like specialized organs, highly differentiated networks cannot function standalone; they need each other. That makes cross-network communication a growth industry.

One honest boundary. Fragmentation guarantees the *edges*; it does not by itself guarantee *capture*. Forks reconnect to their parents through many free channels — a citation, a link, a follow, a shared word — that carry no toll. The monetizable subset is the edges where something of value must actually *settle*: a license, a payment, a scarce credential, a matched order. The free seams are not a loss; they are the funnel that generates the paid ones. Our job is to position on the value-bearing seams, and let the cultural umbilical do the work of creating them.

6 Service, not capture

Capture cannot be the goal, and this is a structural claim, not a slogan. Enshittification [12] is a capture sequence run over time: good to users until they are locked in, then extraction from users to court business customers until *they* are locked in, then extraction from business customers for oneself. Each stage is capture enabling the next extraction.

The reason a toll drifts toward extraction is that it is *decoupled from quality*: it collects whether or not the connector is any good, so nothing punishes decline. A discovery *service* is recoupled to quality: because we have open-sourced the substrate and left the exit open, the moment the service stops helping people find each other, they route around it. An open exit is not a weakness we tolerate; it is the mechanism that forces us to stay good. Capture works by closing exits. We do the opposite, and then earn our position by being the best at discovery rather than the only option.

There is exactly one way this self-correction fails, and it is the way the best discovery service ever built failed. Web search rotted when the party who *pays* (advertisers) stopped being the

party *served* (searchers), and "pay to be surfaced" re-coupled revenue to something other than match quality. This gives us the first of two hard invariants, stated in §10.

7 The differentiator: behavioral query, generated discovery

Discovery is where a connector earns its keep, and §2–§3 have already supplied the foundation no relational engine has: spatial-behavioral types — the third security layer — queried through Rholang's for-comprehension. Because we can ask for processes that will *behave* a certain way, cross-shard search operates over a strictly larger space than data-value search, and because OSLF generates that query logic per language, every service inherits it rather than building it.

The moat here is not the engine. Rholang and OSLF are open source and therefore forkable; search engines were never defended by their algorithm but by their crawl and their click data — coverage and interaction signal, both of which compound and resist forking. F1R3FLY's discovery moat is therefore *reach* (connected to the most shards, seeing the most corpus) and *accumulated signal*. Level-two discovery (inside a community) is the training data that makes level-three discovery (across communities) good: the levels are not three sizes of the same thing; the lower one feeds the higher.

8 Opening sentiment

The like button was a genius move: it reified sentiment. But it left most of the value untouched, because it *closed* what it reified. A like is a scalar, fixed, and owned — legible to the platform, un-ownable by the community. Yet community formation is language formation: communities differentiate themselves by repurposing language, exploiting ambiguity, evolving their own meanings. A closed sentiment primitive forbids the very thing communities exist to do.

So we open sentiment. Color is a good seed because it is a *projective* instrument: ask whether a song, a video, or an article feels red or blue or green, and people have an opinion without knowing why they have it. Opinions will not always line up — and that is the feature, not the bug. The arousal-like dimension of color is fairly stable across people (which lets aggregate queries cohere), while hue-specific meaning is culturally variable (which lets communities build their own affective dialects). Color is only the first dimension; the full panoply recruits the other senses to associate sentiment with digital artifacts, with syntax and semantics that emerge and evolve.

This is the fork-and-connect thesis applied to meaning itself. Sentiment languages fork as communities do, stay non-independent of their parents as arguments do, and the value — and the difficulty — is again in the seams: translating one community's affective vocabulary into another's well enough to discover across them. The reference frame becomes a parameter of the query: "indigo-purple" is never absolute, always indigo-purple *to whom*. Cross-frame queries are translations, lossy the way natural-language translation is lossy, and useful the same way.

The hybrid query, and why it has power

The new capability is a query with an objective and a subjective half:

> *Find all songs near 180 bpm using electric guitar, keyboard, drums, and bass, no vocals, only ambiguous modes (Dorian, Mixolydian; not Ionian or Aeolian), that are indigo through purple.*
>
> *Find all presenters at AGI-2026 who spoke about world models and were blue-green.*
>
> *Find all climate legislation for the Florida wetlands that is red.*

The objective half (tempo, mode, instrumentation) is deterministic feature extraction — dense and nearly free. The subjective half (indigo through purple) has power *precisely because it is orthogonal to* the objective half. If purpleness were recoverable from tempo and mode, we would not need it; the join carries information exactly because the affective gestalt is the residue no feature extractor reaches. This is not SQL with a mood column bolted on. It is a join across two irreducibly different kinds of predicate, where the subjective one carries what the objective one structurally cannot.

These lift to every service above F1R3FLY: *find the PFs in this price range, of this type, that are lavender to pink* (objective price via the PJES denotation, objective type, subjective ink); *find the artists whose last performances made their audiences feel golden* (a pure subjective aggregate). Neither service *built* the discovery. OSLF emitted it.

9 The supply flywheel: storage is the inverse of query

A discovery engine with nothing to discover is a promise, not a product. The supply side is not a second system: as §3 noted, `channel!(data)` makes it easy to put anything anywhere, and it is the inverse of the for-comprehension. Write and read are one primitive seen from two sides. Better query is a reason to store; more stored is better query.

Two consequences compound this. First, probing for *frame-local* sentiment gives a reason to replicate the same asset across several shards — and the only reason to do that is to eventually *compare* frames, which is a cross-shard query by construction. Replication manufactures the demand for the seam traffic we earn on. (Honestly: replication creates *latent* edges that fire when someone wants the comparison; the fork thesis is why that demand is reliably present. And the economics push the heavy blob content-addressed off-shard while the light, queryable projection — behavioral type, price, ink, pointer — is what replicates.)

Second, spatial-behavioral types make F1R3FLY a good home for assets that are natively executable and composable. We should be honest about GitHub: search has almost never decided where code lives — the dependency graph, CI, and collaborators do, and code is the highest-switching-cost asset there is. The real distinction is not finer search; it is that GitHub stores code as *dead text* while F1R3FLY stores it as *live process* one can query by behavior, compose, and price. For an economic model in R, or a physics or biological model, the gap between "a file describing a model" and "a running model others can probe, fork, and compose" is the whole value. So we do not fight GitHub for code-you-store; we win the asset classes that are natively executable-and-composable and have no incumbent with lock-in — PFs, inked artifacts, models-as-live-process.

> The flywheel is incentive-compatible because every motive is *local first*. Nobody stores or inks *for* cross-shard discovery — that would be a free-rider problem. A community brings its assets because it values them, inks them for its own identity and search, and stands up a shard to introduce those assets to itself or invite peer review. The cross-shard corpus is the *byproduct* of activity that was already individually rational. The seam value is an emergent dividend of self-interested local behavior, so it accrues whether or not anyone is trying to produce it — the same reason intranets, built for local value, ended up worth connecting.

10 Two invariants that keep it honest

Everything above depends on two constraints. Break either and the capture engine reassembles itself inside the service.

1. **Payer = served.** Monetize *realized match value*, never surfacing priority. The moment a maker can pay to be found ahead of a better match, revenue decouples from quality and the service rots the way web search did. Our PPF machinery already prices realized value via the PJES denotation [8, 9] over the conditional demand tree, so an ad-valorem fee on delivered value is natural — and it aligns revenue with value actually delivered, rather than a flat per-message fee that is regressive on small matches and captures no upside. In a two-sided market both sides want good matching, but one side always wants to buy its way up the ranking. That door must be welded shut by construction.

2. **Prediction suggests, never authors.** The objective and subjective halves of a hybrid query have opposite economics: features are cheap, dense, machine-supplied; sentiment is expensive, sparse, human-supplied. Early coverage is thin, and the tempting fix is to *predict* the sentiment from the features and backfill. Do that and the machine is authoring the community's feelings instead of recording them — a recommender wearing a crayon, and exactly the closed thing we opened. A model may *propose* "you might feel this is purple"; only a human's mark counts as data. We pay for coverage in authenticity, deliberately.

11 Where the moat actually sits

It is worth being explicit, layer by layer, because the strength is not where the fee is easiest to place.

Substrate (security + scaling). This is the entry wedge and a genuine moat while the lead holds: Byzantine security with crash-tolerant scaling, grounded in the rho calculus, is not something the incumbents can retrofit — their architectures assume the ambient authority we have removed. The durable form of this advantage is *standard-setting*: being the substrate everyone coordinates on.

Transport interconnect (connector shard). The layer most exposed to a free fork, because we open-source the shard and undifferentiated transport historically commoditizes toward cost (internet peering went settlement-free; SMTP and ActivityPub carry no toll). We expect competition here and welcome it. The defensible position is protocol-level fee accrual to a token or treasury rather than a company toll, so the interconnect can be federated and credibly neutral while still capturing value — which also blunts "just fork a free one," because forking away from a neutral standard costs you the network, not a rent-seeker.

Discovery (behavioral query + sentiment). Defended by reach and compounding signal, not by the open-source engine. This is the layer the sentiment work strengthens: the unforkable, use-earned signal that makes cross-shard discovery better the more it is used.

Application-layer matching (PF brokers, Our Thing). The strongest toll, because it taxes matched, two-sided *liquidity*, and liquidity cannot be forked — only code can. Even in the Bitcoin-fork example the capture went to the exchanges that aggregated the flow, not to whoever moved the packets.

12 What we still owe ourselves (open questions)

The case is strong. It is not yet a spreadsheet, and honesty requires naming what remains to be quantified.

- **Realized cross-boundary fraction.** We tax what crosses community lines. The model lives on cross-boundary transaction *frequency*, not shard count. We should form a per-service hypothesis about what share of activity is genuinely inter-shard, and whether replication-for-sentiment realizes latent edges fast enough to matter.

- **Take-rate stacking.** A cross-shard match may bear a transport fee *and* a broker fee, possibly a third at the content layer. Since we forgo endpoint lock-in, the *total* take, seen from the user, must stay genuinely low. We should model it as one stacked number, not as independent "small" fees each layer tells itself are negligible.

- **Local vs. global network effects.** If sentiment and liquidity cluster regionally, revenue is many overlapping local basins rather than one Metcalfe-global network. That is *more* defensible (a rival must out-liquidity every cluster) but changes the growth math to cluster-by-cluster — consistent with our own "one community at a time" thesis, and worth planning for explicitly.

- **Governance of the chokepoint.** A fee-bearing interconnect sits awkwardly with communities that chose sovereign shards for autonomy. Protocol-level accrual and federation are the likely resolution; the details are unsettled and should be designed, not assumed.

- **Connector competition, and defending the lead.** We expect connector shards to become a competitive market. The open question is how fast the differentiation of §7–§8, infrastructure velocity (automatic shard splitting and its successors), and the systems-integrator distribution channel (Tata and others like it) can convert a first-mover lead into a compounding one before the transport layer commoditizes underneath us.

- **Sentiment cold-start and dimension tagging.** Grounded dimensions (arousal-like axes, cross-modal reflexes) translate across communities but differentiate weakly; idiosyncratic dimensions differentiate strongly but barely translate. Each sensory dimension should be tagged by where it falls on that trade, so the query planner and translation layer know which signals cross a seam and which stop at the boundary.

13 Close

The entry is security, and the security is real: a substrate on which the authority-escalation attacks that felled Marks & Spencer and Jaguar Land Rover have no path to run, delivered without giving up the hardware scaling that made the clouds indispensable. That is what gets a Fortune 100 CISO to the table. What keeps them — and what turns a security product into a platform business — is that the same rho-calculus foundation yields programmability, storage, behavioral search, and, at the seams between communities, a connector business.

That business retraces the Internet's own path: build local value first in self-contained shards, then let the value of connecting overwhelm the friction, and earn at the connectors that route across boundaries. The demand there is, we think, settled: fragmentation is the growth engine, and it is accelerating. We place the recurring value where it cannot be forked — security lead and standard, compounding signal, matched liquidity — rather than where it can, and we bind it to two invariants that keep service from decaying into capture. We expect company at the connector layer, and we plan to stay ahead of it by being better, not by being the only one.

The deepest reason to believe the strategy is that the pattern does not belong to F1R3FLY alone — it is what every client will run. Assemble a collection of communities; then build the special means of bringing them together. The Internet made that pattern cheap for bare connectivity; F1R3FLY makes it cheap with programmability, storage, search, and security included, so a client builds only the connector that is its actual business. The Artificial Superintelligence Alliance saw it, and is building ASIChain; Boring Financial saw it, and is building gold-backed digital assets; Weve saw it, and is building Plausible Fiction — three clients, three unrelated domains, one pattern. And it reaches enterprises through the integrators who already carry them: Tata, which absorbed the full cost of the M&S and Jaguar Land Rover breaches, is landed

as the channel to deliver the substrate on which those breaches could not have run. The form fits job markets, content-creation markets, logistics, and whatever forks off them next. That is the case, honestly made. The remaining work is to put numbers to the seams.

References

[1] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[2] L. G. Meredith and M. Radestock. Namespace logic: a logic for a reflective higher-order calculus. In *Trustworthy Global Computing (TGC)*, LNCS 3705, pages 353–369. Springer, 2005.

[3] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[4] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[5] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.

[6] M. Castro and B. Liskov. Practical Byzantine fault tolerance. In *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.

[7] S. Nakamoto. Bitcoin: a peer-to-peer electronic cash system. 2008. https://bitcoin.org/bitcoin.pdf

[8] S. Peyton Jones, J.-M. Eber, and J. Seward. Composing contracts: an adventure in financial engineering. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000.

[9] F1R3FLY.io. Financial contracts in the cost-accounted rho calculus (FinRho). F1R3FLY Publications. https://github.com/F1R3FLY-io/publications/blob/main/financial-contracts/finrho.pdf

[10] F1R3FLY.io. Plausible Fiction. F1R3FLY Publications (with D. Spivak, Topos Institute). https://github.com/F1R3FLY-io/publications/blob/main/plausible-fiction/plausible-fiction.pdf

[11] F1R3FLY.io. Priced Plausible Fiction. F1R3FLY Publications. https://github.com/F1R3FLY-io/publications/blob/main/plausible-fiction/priced-pf.pdf

[12] C. Doctorow. The 'enshittification' of TikTok. *Wired*, January 2023.

[13] P. Scholze and D. Clausen. Lectures on condensed mathematics. Lecture notes, University of Bonn, 2019–2020.

[14] M. Kapko. M&S warns April cyberattack will cut $400 million from profits. *Cybersecurity Dive*, May 2025. https://www.cybersecuritydive.com/news/ms-cyberattack-400-million/748710/

[15] M&S chairman calls for mandatory disclosure of material cyberattacks. *Cybersecurity Dive*, July 2025. https://www.cybersecuritydive.com/news/ms-chairman-mandatory-disclosure-material-cyberattacks/752584/

[16] Z. Whittaker. UK government bails out Jaguar Land Rover with £1.5B loan after hack disrupts vehicle production for weeks. *TechCrunch*, September 2025. https://techcrunch.com/2025/09/29/uk-government-bails-out-jaguar-land-rover-with-1-5b-loan-after-hack-disrupts-vehicle-pr

[17] Jaguar Land Rover cyber attack: economic impact modelled at £1.9 billion by the Cyber Monitoring Centre. Industry reporting, October 2025–January 2026.